Bias-Corrected Meta-Analysis Model

Pablo Emilio Verde

01 September 2020

Contents

1 Introduction **2**
1.1 Table 1 . . . 2
1.2 Analysis of the introductory example . . . 3
1.2.1 Bayesian random-effects meta-analysis model . . . 3
1.2.1.1 Analysis of RCTs only . . . 4
1.2.1.2 Analysis of OSs only . . . 5
1.2.2 Bias-Corrected meta-analysis model . . . 5
1.3 Figure 1 . . . 8

2 Section 2.1: Priors of Hyper-parameters **8**

3 Section 3: Real data examples **9**
3.1 Generalized evidence synthesis of IPD . . . 9
3.1.1 Hyper-priors . . . 9
3.1.2 Figure 3 . . . 9
3.1.3 Effective number of studies N^+ . . . 9
3.1.4 Comparison between the Bayesian random-effects meta-analysis and the BC model . . . 10
3.1.5 Figure 4: diagnostic plot . . . 11
3.1.6 Effective number of studies N^+ . . . 13
3.2 Meta-analysis of RCTs with varying quality: heart disease and bone marrow stem cells treatment 14
3.2.1 Simple Bayesian Meta-Analysis . . . 14
3.2.2 Bias-corrected analysis . . . 15
3.2.3 Figure 5 . . . 17
3.2.4 Diagnostics . . . 17
3.2.5 Study level results . . . 18
3.2.6 Model to link discrepancies and probability of bias . . . 20

1 Introduction

1.1 Table 1

```
data("ppvipd")

dat.ipv.all = ppvipd

# Source: Table 2 @Falkenhorst et al. (2017)
dat.ipv.all$Risk = c("Unclear", "Low", "Low", "High", "Low", "Low", "High",
                     "Low", "Low","Low", "Low")

dat.ipv.all[, c("name", "design","TE", "seTE", "n.c", "n.v","Risk")]
```

```
##                      name       design         TE      seTE    n.c    n.v
## 1          Honkanen 1999          RCT -0.9722612 0.8367229  18488  19549
## 2          Maruyama 2010          RCT -1.9406631 1.5124362   1149   1140
## 3          Ortqvist 1998          RCT -1.5178076 1.0965467    873    793
## 4           Hechter 2012       cohort -1.0498000 0.8998000  27320   3962
## 5           Jackson 2003       cohort -0.5852000 0.2615000  42977  84203
## 6      Ochoa-Gondar 2014       cohort -0.9676000 0.7584000  30000  28662
## 7              Tsai 2015       cohort -1.4271000 0.3342000 229181 229181
## 8     Vila-Corcoles 2006       cohort -0.5108000 0.5161000  16504  17401
## 9         Dominguez 2005 case-control -1.1882000 0.2687000    447    149
## 10 Leventer-Roberts 2015 case-control -0.5463000 0.1712000    848    212
## 11    Vila-Corcoles 2009 case-control -1.0850000 0.3416000    188     94
##       Risk
## 1  Unclear
## 2      Low
## 3      Low
## 4     High
## 5      Low
## 6      Low
## 7     High
## 8      Low
## 9      Low
## 10     Low
## 11     Low
```

```
#write.table(dat.ipv.all[, c("name", "design","TE", "seTE", "n.c", "n.v","Risk")],
#            "Table_1.txt", sep = "  &  ", row.names = FALSE)
```

1.2 Analysis of the introductory example

We set the seed of the random numbers at 2020 (Year):

```
set.seed(2020)
```

For each run of the jags() function we left the jags.seed at the default value: "jags.seed=123".

```
# Invlogit
invlogit = function(x)
{
  1/(1 + exp(-x))
}
```

1.2.1 Bayesian random-effects meta-analysis model

```
library(R2jags)

# Bayesian Random Effects Model ......................

# Model in BUGS
cat(
  "model
  {
   for( i in 1 : N ) {
  # Likelihood of theta[i] .......................

            y[i] ~ dnorm(theta[i], pre.y[i])
       pre.y[i] <- pow(se.y[i], -2)

  # Simple random effects .......................

     theta[i] ~ dnorm(mu, prec.tau)
}

# Priors for hyper-parameters ....................................

# Variance components ...

tau <- 1/sqrt(prec.tau)

prec.tau ~ dscaled.gamma(scale.sigma.between,
                        df.scale.between)        # spike: tau is small (unbias mu)

# Prior for mu
mu ~ dnorm(0.0, 0.001)

# Posterior predictive distribution
theta.new ~ dnorm(mu, prec.tau)
  }",
  file = "meta_bayes.bugs")

#############################################################

# Data model
   y = dat.ipv.all$TE
se.y = dat.ipv.all$seTE
   N = length(y)

scale.sigma.between = 0.5
df.scale.between = 1
```

```
# This list describes the data used by the BUGS script.
data.meta.bayes <- list ("y", "se.y", "N",
                 "scale.sigma.between",
                 "df.scale.between")

# List of parameters
par.meta.bayes  <- c("theta",
                     "mu",
                     "theta.new",
                     "tau")
```

```
# RCTs

##########################################################

   y = dat.ipv.all$TE[dat.ipv.all$design=="RCT"]
se.y = dat.ipv.all$seTE[dat.ipv.all$design=="RCT"]
   N = length(y)
   N
```

1.2.1.1 Analysis of RCTs only

```
## [1] 3
```

```
# Call with OpenBUGS
m.rct <- jags(data.meta.bayes,
             inits = NULL,
             parameters.to.save = par.meta.bayes,
             model = "meta_bayes.bugs",
             n.chains = 4,
             n.iter = 50000,
             n.thin = 1,
             n.burnin = 20000,
             DIC = TRUE)
```

```
## module glm loaded
```

```
## Compiling model graph
##    Resolving undeclared variables
##    Allocating nodes
## Graph information:
##    Observed stochastic nodes: 3
##    Unobserved stochastic nodes: 6
##    Total graph size: 25
##
## Initializing model
```

```
attach.jags(m.rct, overwrite = TRUE)
mu.rct = mu
```

```
# OS

##########################################################
```

```
   y = dat.ipv.all$TE[dat.ipv.all$design!="RCT"]
se.y = dat.ipv.all$seTE[dat.ipv.all$design!="RCT"]
   N = length(y)
   N
```

1.2.1.2 Analysis of OSs only

```
## [1] 8
```

```
# Call with OpenBUGS
m.os <- jags(data.meta.bayes,
             inits = NULL,
             parameters.to.save = par.meta.bayes,
             model = "meta_bayes.bugs",
             n.chains = 4,
             n.iter = 50000,
             n.thin = 1,
             n.burnin = 20000,
             DIC = TRUE)
```

```
## Compiling model graph
##    Resolving undeclared variables
##    Allocating nodes
## Graph information:
##    Observed stochastic nodes: 8
##    Unobserved stochastic nodes: 11
##    Total graph size: 45
##
## Initializing model
```

```
attach.jags(m.os, overwrite = TRUE)
mu.os = mu
```

1.2.2 Bias-Corrected meta-analysis model

```
# The Bias Corrected Model ........................................

# Model in BUGS
cat(
  "model
  {
   for( i in 1 : N ) {
  # Likelihood of theta[i] ........................................

            y[i] ~ dnorm(theta.bc[i], pre.y[i])
       pre.y[i] <- pow(se.y[i], -2)

# Mixture heterocedastic random effects ........................

      theta.bc[i] <- theta[i]*(1-I[i]) + theta.bias[i]*I[i]
             I[i] <- T[i] -1
          theta[i] ~ dnorm(mu[1], prec.tau[i])
     theta.bias[i] ~ dnorm(mu[2], prec.tau[i])

            T[i] ~ dcat(p.bias[1:2])
     prec.tau[i] <- inv.var[T[i]] * w[T[i],i]  #Slash parametrization
          w[1,i] <- 1

      # slash distribution ...
```

```
      w[2,i] ~ dbeta(nu, 1)

}

nu <- 1/2

# Priors for hyper-parameters ...............................

# Prior of probability classes

p.bias[2] ~ dbeta(alpha.bias, beta.bias)
p.bias[1] <- 1 - p.bias[2]

# Variance components ...

tau <- 1/sqrt(inv.var[1])

inv.var[1] ~ dscaled.gamma(scale.sigma.between,
                df.scale.between)

inv.var[2] <- inv.var[1]

# Prior for mu
mu[1] ~ dnorm(0.0, 0.01)
B ~ dunif(0, B.max)
mu[2] <- mu[1] + B

  }",
  file = "BC.bugs")

##########################################################

   y = sort(dat.ipv.all$TE)
se.y = dat.ipv.all$seTE[order(dat.ipv.all$TE)]
   N = length(y)

#T is the index that allocates each study to one of the two components
   T = rep(NA, N)
T[1] = 1
T[N] = 2

scale.sigma.between = 0.5
df.scale.between = 1

# Probability to be biased
# Number of OS = 8
# Total = 11

# The median of the prior is: (Number of OS - 1) / Total = 7/11
# and the 90th percentile of the prior is: Number of OS  / Total = 8/11
#
# The idea is to limit the number of biased studies up to the number of OS
# and open the posibility to include unbiased studies, which are OS.

library(LearnBayes)
```

```
##
## Attaching package: 'LearnBayes'


## The following object is masked from 'package:extraDistr':
##
##     rdirichlet
```

```
quantile1=list(p = 0.5, x = 7/11)
quantile2=list(p = 0.9, x = 8/11)

beta.par = beta.select(quantile1, quantile2)
alpha.bias = beta.par[1]
beta.bias = beta.par[2]

# Non-informative prior for the probability of bias
# alpha.bias = 1
# beta.bias = 1

#alpha.bias = beta.bias = 1

 B.max = 10

# This list describes the data used by the BUGS script.
data.BC <- list ("y", "se.y", "N", "T",
                 "scale.sigma.between",
                 "df.scale.between",
                 "alpha.bias",
                 "beta.bias",
                 "B.max")

# List of parameters
par.BC  <- c("theta.bc",
             "theta",
             "theta.bias",
             "mu",
             "tau",
             "p.bias",
             "I",
             "w",
             "B")
```

```
# Call with OpenBUGS
bc.1 <- jags(data.BC,
             inits = NULL,
             parameters.to.save = par.BC,
             model = "BC.bugs",
             n.chains = 4,
             n.iter = 50000,
             n.thin = 4,
             n.burnin = 20000,
             DIC = TRUE)
```

```
## Compiling model graph
##    Resolving undeclared variables
##    Allocating nodes
## Graph information:
##    Observed stochastic nodes: 13
##    Unobserved stochastic nodes: 46
##    Total graph size: 172
##
## Initializing model
```

```
attach.jags(bc.1, overwrite = TRUE)

mu.bc = mu[,1]
```

1.3 Figure 1

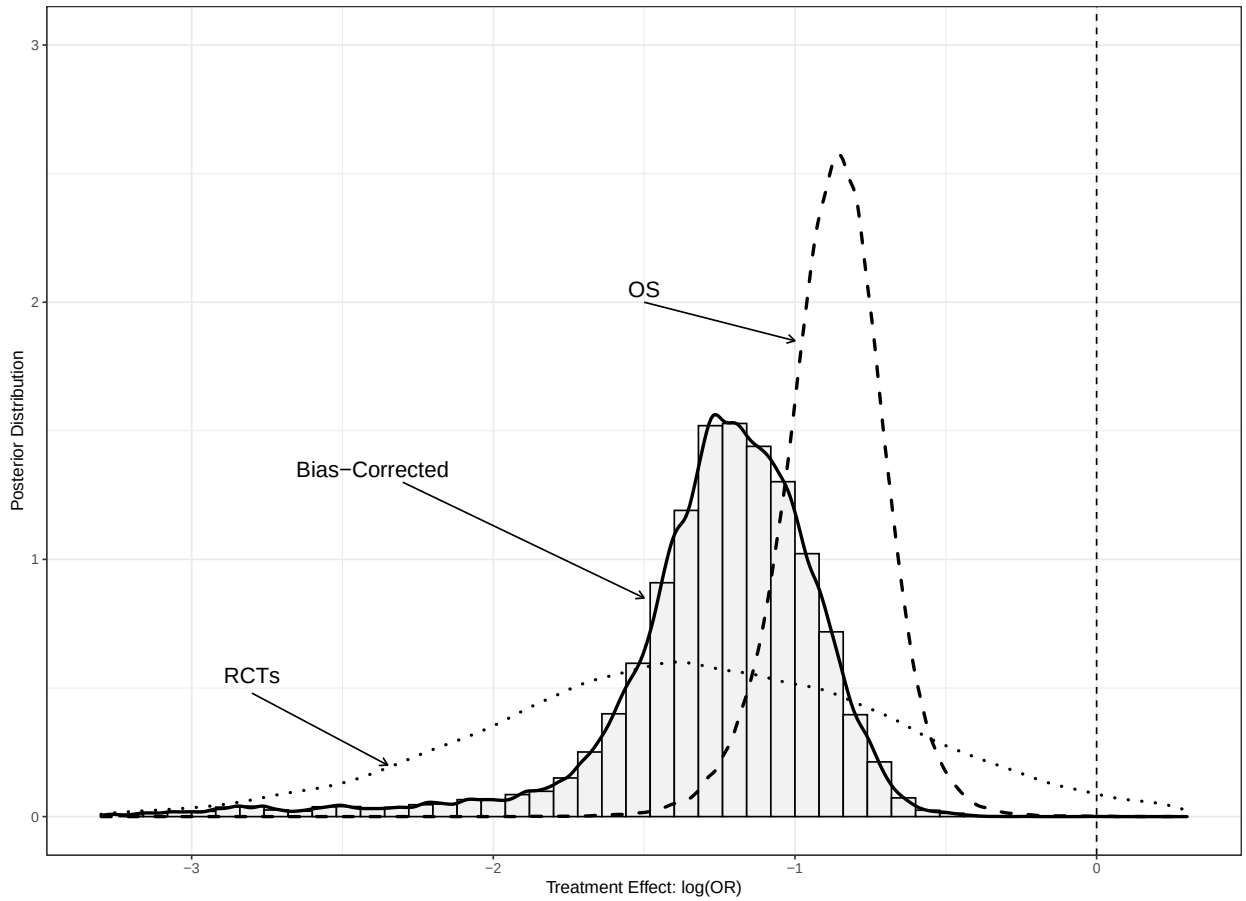


Figure 1: *Bias corrected posterior distribution of the efficacy/effectiveness of PPV23 vaccination in preventing IPD (invasive pneumococcal disease).*

2 Section 2.1: Priors of Hyper-parameters

Notes on the Half-Cauchy distribution:

```
S = 0.5

# Probability that tau > 0.5 for S = 0.5
1-phcauchy(0.5, sigma = 0.5)
```

```
## [1] 0.5
```

```
# Probability that tau > 4*0.5 for S = 0.5
round(1-phcauchy(0.5*4, sigma = 0.5),3)
```

```
## [1] 0.156
```

3 Section 3: Real data examples

3.1 Generalized evidence synthesis of IPD

3.1.1 Hyper-priors

Hyper-parameters for $\pi_{bias} \sim Beta(a_0, a_1)$:

```
alpha.bias
```

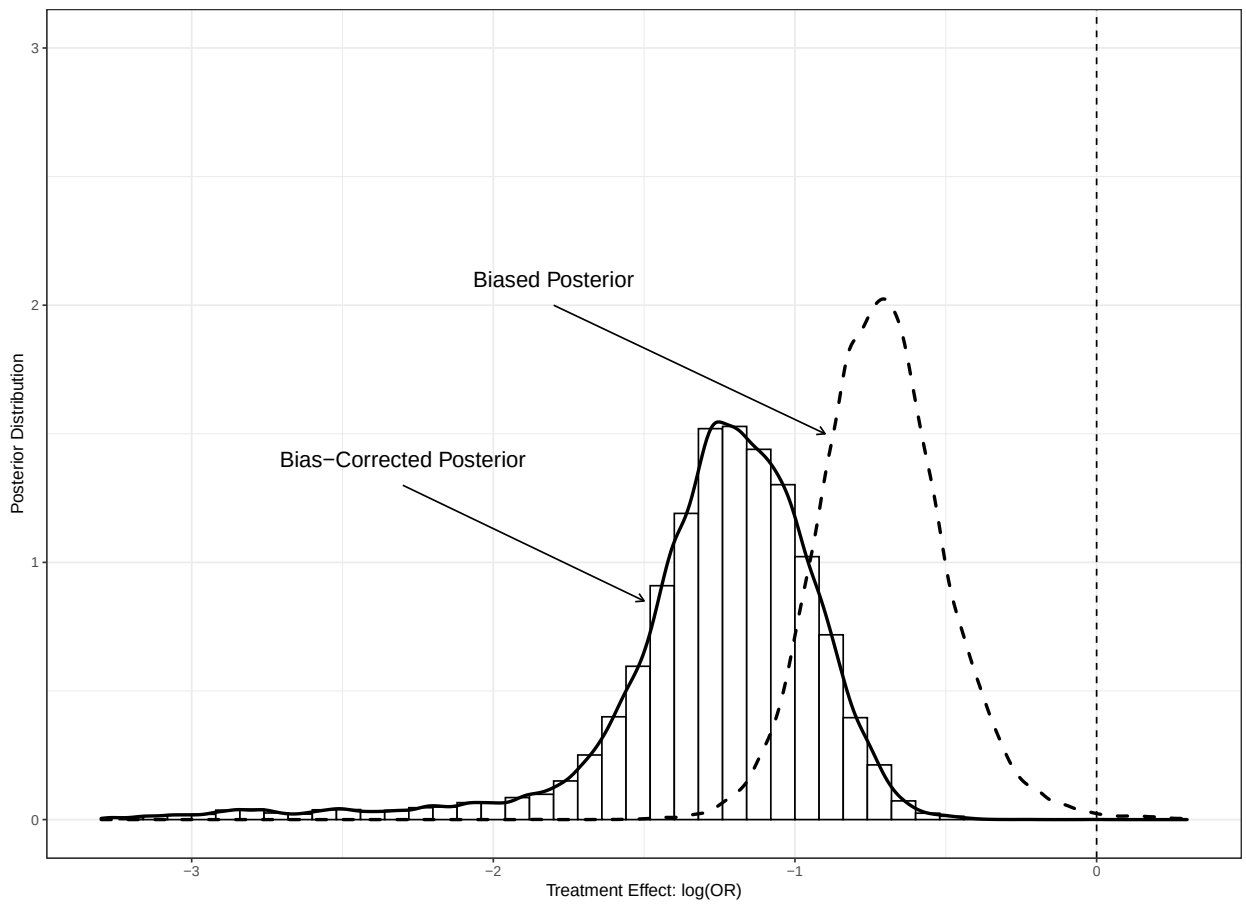
```
## [1] 26.83
```

```
beta.bias
```

```
## [1] 15.48
```

3.1.2 Figure 3

```
attach.jags(bc.1, overwrite = TRUE)
```



3.1.3 Effective number of studies N^+

```
# N = number of studies
# 8 are OS
N
```

```
## [1] 11
```

```
# Prior probability of bias
8/N
```

```
## [1] 0.7272727
```

```
# Posterior probability of bias
attach.jags(bc.1, overwrite = TRUE)

p.bias.1 = p.bias[,2]

round(mean(p.bias.1),2)
```

```
## [1] 0.62
```

```
# Effective number of studies
round(mean((1-p.bias.1)*N),2)
```

```
## [1] 4.13
```

3.1.4 Comparison between the Bayesian random-effects meta-analysis and the BC model

```
round(quantile(mu.rct, probs = c(0.025, 0.975)),2)
```

```
##  2.5% 97.5%
## -2.78  0.10
```

```
round(mean(mu.rct),2)
```

```
## [1] -1.34
```

```
RCT.interval = quantile(mu.rct, probs = c(0.025, 0.975))

Length.RCT.interval = RCT.interval[2] - RCT.interval[1]

round(Length.RCT.interval,2)
```

```
## 97.5%
##  2.88
```

```
round(mean(mu.bc),2)
```

```
## [1] -1.35
```

```
BC.interval = quantile(mu.bc, probs = c(0.025, 0.975))

round(BC.interval, 2)
```

```
##  2.5% 97.5%
## -3.42 -0.76
```

```
Length.BC.interval = BC.interval[2] - BC.interval[1]

round(Length.BC.interval,2)
```

```
## 97.5%
##  2.66
```

```
# 3.1
# Comparison between RCT and BC intervals

round(Length.RCT.interval/Length.BC.interval*10,2)
```

```
## 97.5%
## 10.82
```

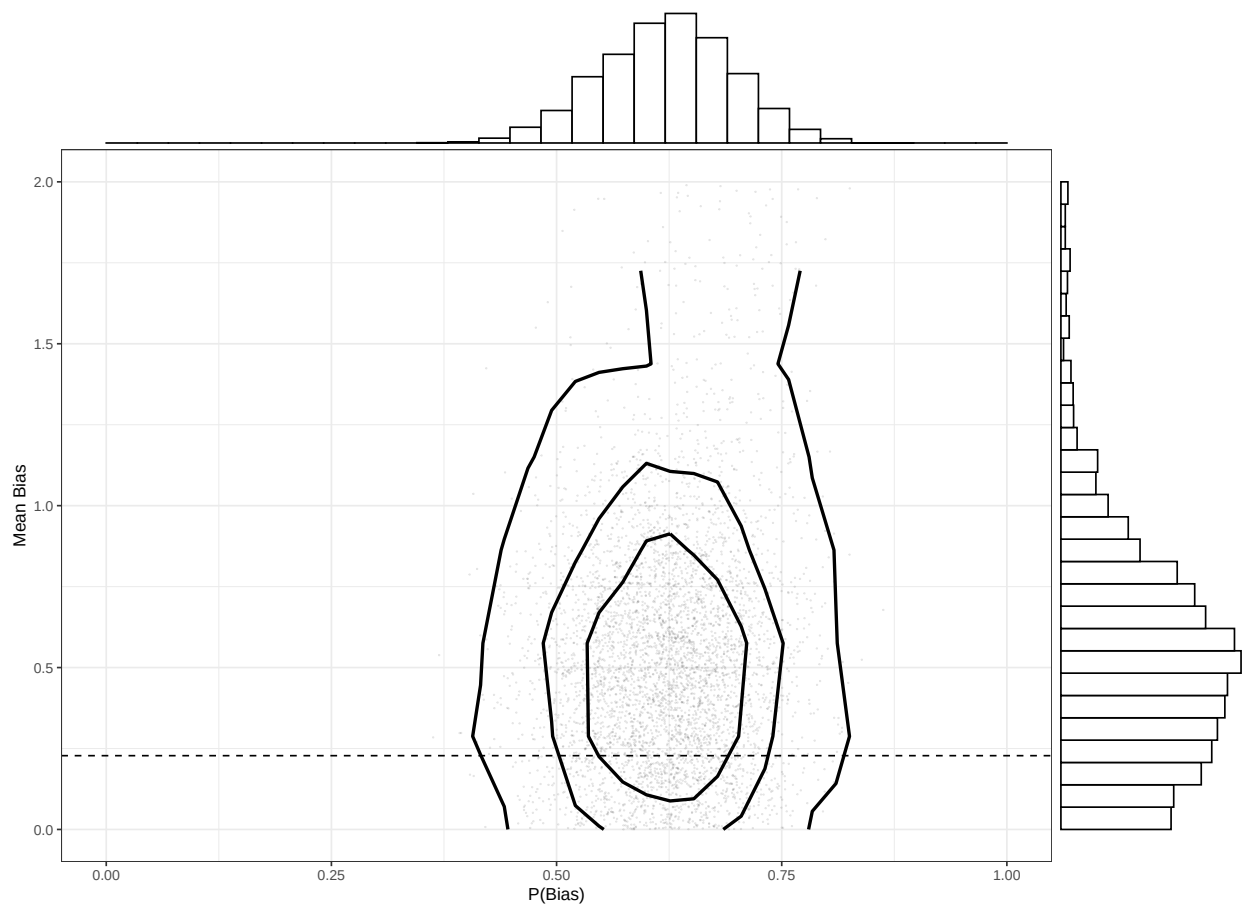
3.1.5 Figure 4: diagnostic plot

Diagnostic function to display "bias" against "probability or OR of bias"

```
library(ggExtra)

# Diagnostic function to display "bias" against "probability or OR of bias"
plot.delta.pi <- function(pi.bias, mu, tau,
                          level = c(0.5, 0.75, 0.95),
                          limits.x = c(0, 1),
                          limits.y = c(0, 10),
                          label.x = "P(Bias)",
                          label.y = "Mean Bias",
                          kde2d.n = 25,
                          color.line = "black",
                          title = paste("Posterior Contours (50%, 75% and 95%)"),
                          #marginals = TRUE,
                          bin.hist = 30,
                          color.hist = "white",
                          S = 5000,
                          color.data.points = "black",
                          alpha.data.points = 0.1,
                          ...)
{

  p.bias.1 = pi.bias
  delta.1 =   mu[,2] -mu[,1]
  tau.1 = tau

  cut.point = 2*mean(tau)

  dat.post = data.frame(x=p.bias.1, y=delta.1)

  dat.post = dat.post[sample(1:S), ]

  # Base plot ..............................................................................
  baseplot = ggplot(dat.post, aes(x = x, y = y)) +
```

```
    geom_point(size=0.01, alpha = alpha.data.points,
               aes(color = "MCMC Samples"), color = color.data.points)+
    scale_x_continuous(name = label.x, limits = limits.x) +
    scale_y_continuous(name = label.y, limits = limits.y) +
  #  ggtitle("Posterior Contours (50%, 75% and 95%)") +
    geom_hline(yintercept=cut.point, linetype="dashed", color = "black")+
    theme_bw()


  # Non-parametric ..........................................................................
  # Estimation of the nonparametric density ...
  x.nopar <- dat.post[ ,1]
  y.nopar <- dat.post[ ,2]

  dens <- kde2d(x.nopar, y.nopar, n = 20)
  dx <- diff(dens$x[1:2])
  dy <- diff(dens$y[1:2])
  sz <- sort(dens$z)
  c1 <- cumsum(sz) * dx * dy
  Levels.nonpar <- approx(c1, sz, xout = 1 - level)$y


  densdf = data.frame(expand.grid(pi.bias = dens$x,
                                  bias = dens$y),
                      dens.z = as.vector(dens$z))

  # Non-parametric ............
  finalplot=baseplot + geom_contour(data = densdf,
                                    aes(pi.bias, bias, z = dens.z),
                                    colour = color.line,
                                    breaks = Levels.nonpar,
                                    lwd =1)

  nparplot = ggMarginal(finalplot, type= "histogram",
                        fill = color.hist,
                        bins = bin.hist)
  return(nparplot)
}
```

```
p.bias.1 = p.bias[,2]
mu.1 =   mu[,1:2]
tau.1 = tau

plot.delta.pi(pi.bias = p.bias.1, mu = mu.1, tau=tau.1,
              limits.x = c(0, 1),
              limits.y = c(0, 2))
```



3.1.6 Effective number of studies N^+

```
N
```

```
## [1] 11
```

```
8/N
```

```
## [1] 0.7272727
```

```
round(mean(p.bias.1),2)
```

```
## [1] 0.62
```

```
round(mean((1-p.bias.1)*N),2)
```

```
## [1] 4.13
```

3.2 Meta-analysis of RCTs with varying quality: heart disease and bone marrow stem cells treatment

```
library(jarbes)
data(stemcells)
```

3.2.1 Simple Bayesian Meta-Analysis

```
   y = sort(stemcells$effect.size)
se.y = stemcells$se.effect[order(y)]
   N = length(y)

scale.sigma.between = 0.5
df.scale.between = 1

# This list describes the data used by the BUGS script.
data.meta.bayes <- list ("y", "se.y", "N",
                 "scale.sigma.between",
                 "df.scale.between")

# List of parameters
par.meta.bayes  <- c("theta",
                     "mu",
                     "theta.new",
                     "tau")

# Call with OpenBUGS
m.simple <- jags(data.meta.bayes,
             inits = NULL,
             parameters.to.save = par.meta.bayes,
             model = "meta_bayes.bugs",
             n.chains = 4,
             n.iter = 50000,
             n.thin = 4,
             n.burnin = 20000,
             DIC = TRUE)
```

```
## Compiling model graph
##    Resolving undeclared variables
##    Allocating nodes
## Graph information:
##    Observed stochastic nodes: 31
##    Unobserved stochastic nodes: 34
##    Total graph size: 134
##
## Initializing model
```

```
attach.jags(m.simple, overwrite = TRUE)

mu.simple = mu
mu.new.simple = theta.new
tau.simple = tau
```

```
# Results of the simple Bayesian meta-analysis of steam cells example

# Pooled mean
round(mean(mu.simple),2)
```

```
## [1] 3.15
```

```r
round(quantile(mu.simple, probs = c(0.025, 0.975)),2)
```

```
##  2.5% 97.5%
##  1.54  4.75
```

```r
# tau
round(mean(tau.simple),2)
```

```
## [1] 3.88
```

```r
round(quantile(tau.simple, probs = c(0.025, 0.975)),2)
```

```
##  2.5% 97.5%
##  2.81  5.30
```

```r
# Predictive
round(mean(mu.new.simple),2)
```

```
## [1] 3.15
```

```r
round(quantile(mu.new.simple, probs = c(0.025, 0.975)),2)
```

```
##  2.5% 97.5%
## -4.86 11.10
```

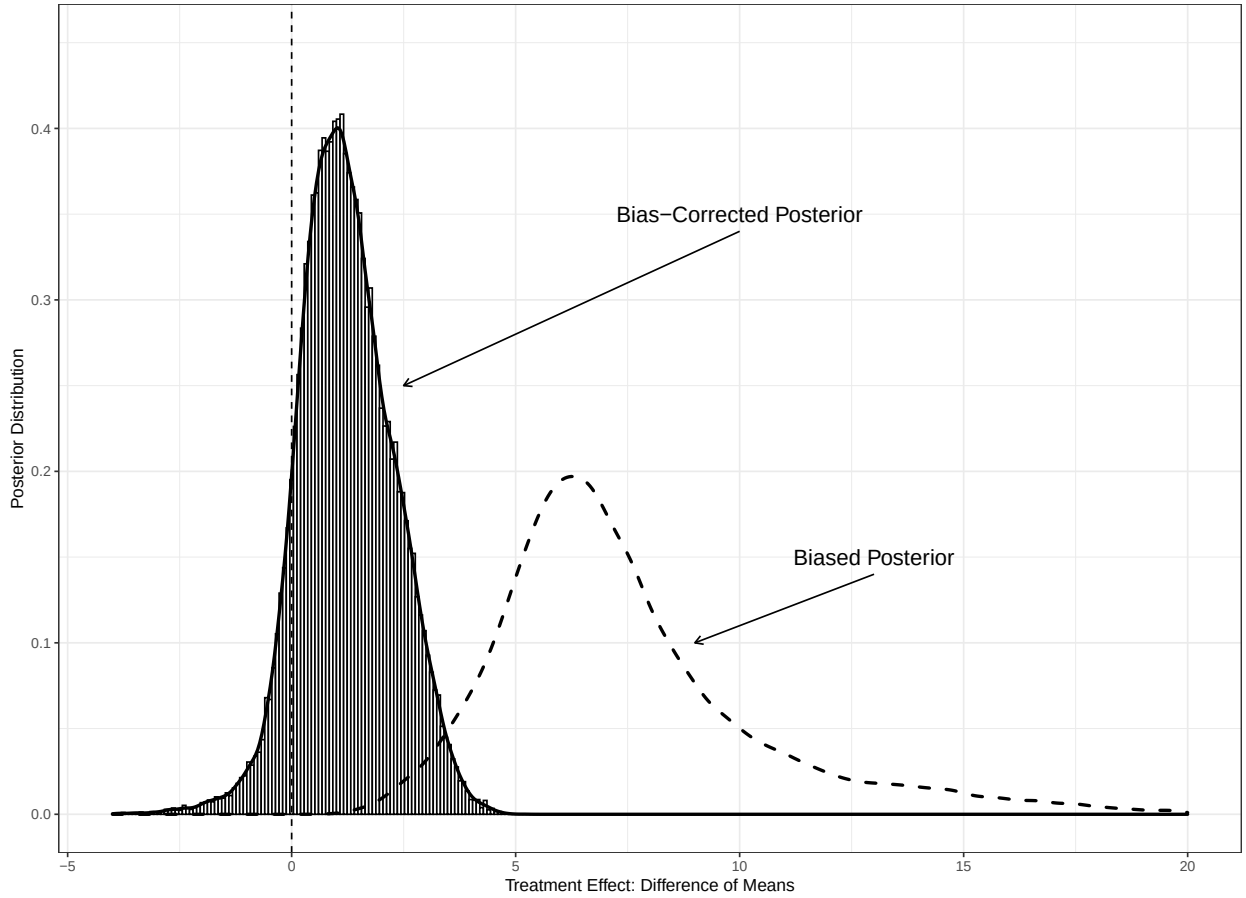
3.2.2 Bias-corrected analysis

```r
   y = sort(stemcells$effect.size)
se.y = stemcells$se.effect[order(y)]
   N = length(y)
   x = stemcells$n.discrep[order(y)]

# T is the index that allocates each observation to one of the two normals.
   T = rep(NA, N)
T[1] = 1
T[N] = 2

scale.sigma.between = 0.5
df.scale.between = 1

# Probability to be biased

alpha.bias = 1
beta.bias =  1

# Bias Upper limit
# Ojo aca el rango es de 20 o mas ...
B.max = 30

# This list describes the data used by the BUGS script.
data.BC <- list ("y", "se.y", "N", "T",
                 "scale.sigma.between",
                 "df.scale.between",
                 "alpha.bias",
                 "beta.bias",
```

```
                 "B.max")

# List of parameters
par.BC  <- c("theta.bc",
             "theta",
             "mu",
             "tau",
             "p.bias",
             "T",
             "w",
             "B")

# Call with OpenBUGS
bc.2 <- jags(data.BC,
             inits = NULL,
             parameters.to.save = par.BC,
             model = "BC.bugs",
             n.chains = 4,
             n.iter = 50000,
             n.thin = 4,
             n.burnin = 2000,
             DIC = TRUE)
```

```
## Compiling model graph
##    Resolving undeclared variables
##    Allocating nodes
## Graph information:
##    Observed stochastic nodes: 33
##    Unobserved stochastic nodes: 126
##    Total graph size: 449
##
## Initializing model
```

3.2.3 Figure 5



Posterior summaries of the pooled unbiased mean:

```
round(mean(mu.bc),2)
```

```
## [1] 1.22
```

```
BC.interval = quantile(mu.bc, probs = c(0.025, 0.975))

round(BC.interval, 2)
```

```
##  2.5% 97.5%
## -0.75  3.30
```

3.2.4 Diagnostics

Probability that a biased component exists:

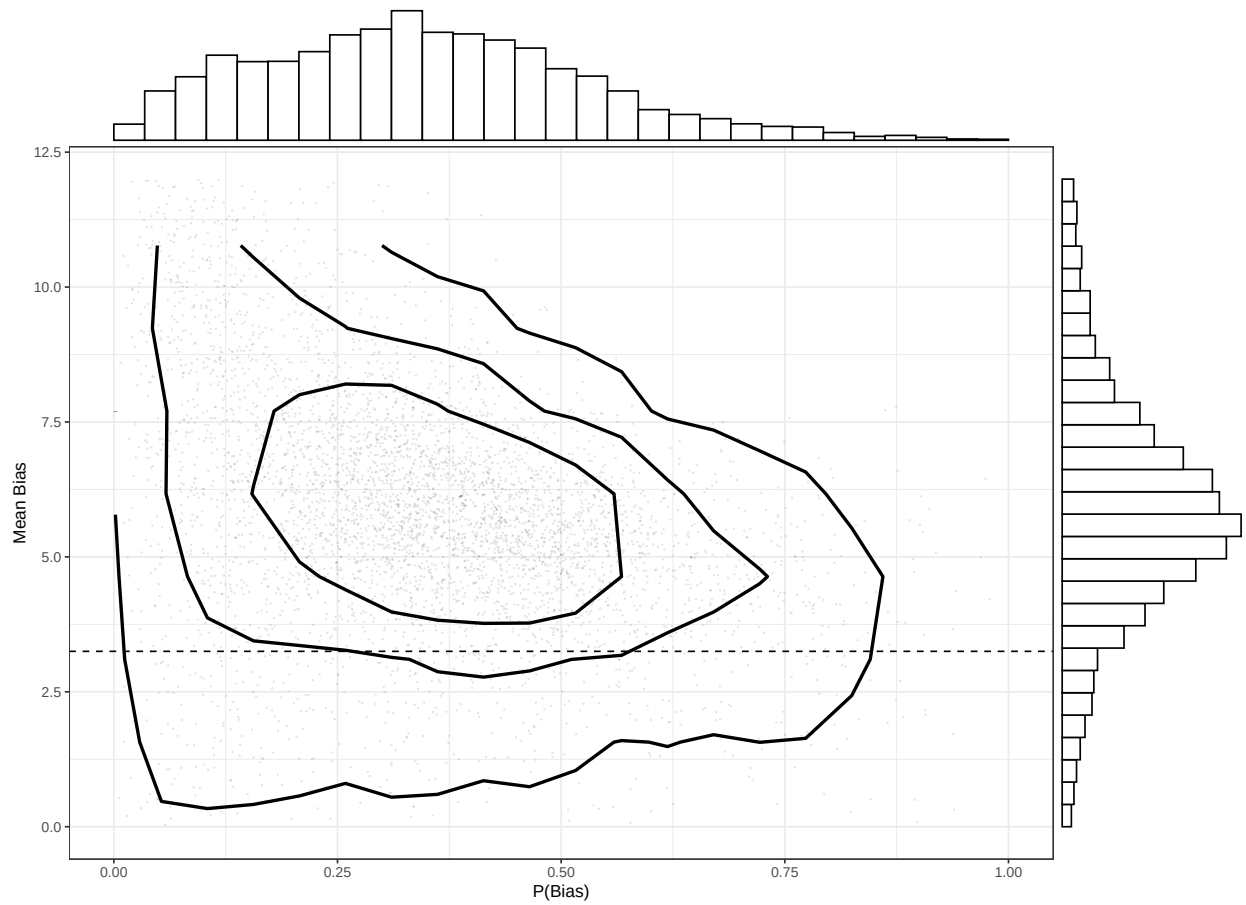
```
attach.jags(bc.2, overwrite = TRUE)

p.bias.1 = p.bias[,2]
mu.1 =   mu[,1:2]
tau.1 = tau
```

```
# Diagnostic two modes
delta.1 =   mu[,2] -mu[,1]
cut.point = 2*mean(tau)

# Posterior probability of having a bias component
round(1-sum(delta.1 < cut.point)/length(delta.1), 3)
```

```
## [1] 0.91
```

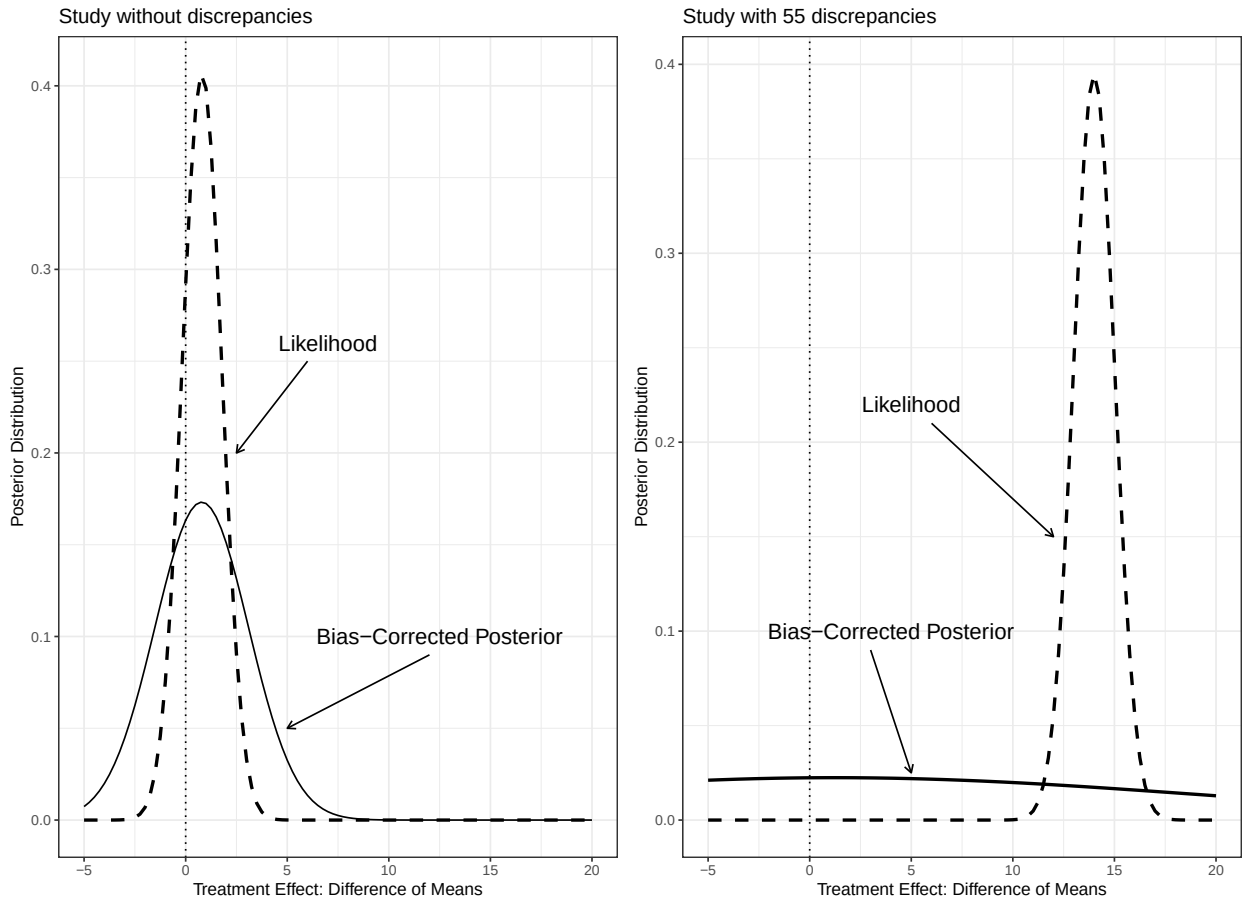
```
plot.delta.pi(pi.bias = p.bias.1, mu = mu.1, tau=tau.1,
              limits.x = c(0, 1),
              limits.y = c(0, 12))
```

```
## Warning: Removed 240 rows containing non-finite values (stat_contour).

## Warning: Removed 240 rows containing non-finite values (stat_contour).

## Warning: Removed 218 rows containing missing values (geom_point).
```



3.2.5 Study level results

```
attach.jags(bc.2, overwrite = TRUE)


df.text.1 <- data.frame(x = c(7, 12.5),
                        y = c( 0.26,  0.1),
                        study.type = c("Likelihood",
                                       "Bias-Corrected Posterior"))

df.text.2 <- data.frame(x = c(5, 4),
                        y = c( 0.22,  0.1),
                        study.type = c("Likelihood",
                                       "Bias-Corrected Posterior"))


p1 = ggplot(data = data.frame(x = c(-5, 20)), aes(x)) +
  stat_function(fun = dnorm, args = list(mean = y[10], sd = se.y[10]), lty=2, lwd = 1) +
  stat_function(fun = dnorm, args = list(mean = mean(theta[,10], lwd = 1),
                                         sd = sd(theta[,10]))) +
  geom_vline(aes(xintercept=0), lty = 3)+
  ylab("Posterior Distribution")+
  ggtitle("Study without discrepancies")+
  xlab("Treatment Effect: Difference of Means")+
  geom_text(data = df.text.1, aes(x=x, y=y, label = study.type), size = 5)+
  annotate(geom = "segment",
           x = 6, y = 0.25,
           xend = 2.5, yend = 0.2,
           arrow = arrow(length = unit(2, "mm")))+
  annotate(geom = "segment",
           x = 12, y = 0.09,
           xend = 5, yend = 0.05,
           arrow = arrow(length = unit(2, "mm")))+
  theme_bw()


p2 = ggplot(data = data.frame(x = c(-5, 20)), aes(x)) +
  stat_function(fun = dnorm,
                args = list(mean = y[31], sd = se.y[31]),
                lty=2, lwd = 1) +
  stat_function(fun = dnorm, args = list(mean = mean(theta[,31]),
                                         sd = sd(theta[,31])), lwd = 1) +
  geom_vline(aes(xintercept=0), lty = 3)+
  ylab("Posterior Distribution")+
  ggtitle("Study with 55 discrepancies")+
  xlab("Treatment Effect: Difference of Means")+
  geom_text(data = df.text.2, aes(x=x, y=y, label = study.type), size = 5)+
  annotate(geom = "segment",
           x = 6, y = 0.21,
           xend = 12, yend = 0.15,
           arrow = arrow(length = unit(2, "mm")))+
  annotate(geom = "segment",
           x = 3, y = 0.09,
           xend = 5, yend = 0.025,
           arrow = arrow(length = unit(2, "mm")))+
  theme_bw()

grid.arrange(p1, p2, nrow = 1)
```



3.2.6 Model to link discrepancies and probability of bias

```
# The Bias Corrected Model ..........................................
# This BUGS scrip contains three alternative models: 1) two different link functions: logit and cloglog
#

# Model in BUGS
cat(
  "model
  {
   for( i in 1 : N ) {
  # Likelihood of theta[i] ........................................

            y[i] ~ dnorm(theta.bc[i], pre.y[i])
       pre.y[i] <- pow(se.y[i], -2)

  # Mixture heterocedastic random effects .........................

     theta.bc[i] ~ dnorm(mu[T[i]], prec.tau[i])
            T[i] ~  dcat(P[i, 1:2])
     logit(P[i, 2]) <- alpha.0 + alpha.1*x[i]
  #  cloglog(P[i, 2]) <- alpha.0 + alpha.1*x[i]

  # Cut-off point models for x ...
  #  logit(P[i, 2]) <- alpha.0 + alpha.1*z[i]
  #  cloglog(P[i, 2]) <- alpha.0 + alpha.1*z[i]
```

```
    z[i] <- step(x[i] - K.x)

   P[i, 1] <- 1 - P[i, 2]

     prec.tau[i] <- inv.var[T[i]] * w[T[i],i]  #Gamma or Slash parametrization
          w[1,i] <- 1

      # slash distribution ...
      w[2,i] ~ dbeta(nu, 1)

   }



nu <- 1/2

# Priors for hyper-parameters ......................................

# Prior of probability of bias
 alpha.0 ~ dnorm(prior.mean.alpha.0, prior.scale.alpha.0)
 alpha.1 ~ dnorm(prior.mean.alpha.1, prior.scale.alpha.1)

# This prior is only needed for a cutoff point of x...
  K.x ~ dunif(0, 60)

# Variance components ...

tau <- 1/sqrt(inv.var[1])

inv.var[1] ~ dscaled.gamma(scale.sigma.between,
                df.scale.between)        # spike: tau is small (unbias mu)

inv.var[2] <- inv.var[1] #* q.weight     # slab: tau is large (bias mu)

# Prior for mu
mu[1] ~ dnorm(0.0, 0.01)
B ~ dunif(0, B.max)
mu[2] <- mu[1] + B

  }",
  file = "PBias.bugs")
```

```
# Data
 y = sort(stemcells$effect.size)
se.y = stemcells$se.effect[order(y)]
   N = length(y)
   x = stemcells$n.discrep[order(y)]

# T is the index that allocates each observation to one of the two normals.
   T = rep(NA, N)
T[1] = 1
T[N] = 2

scale.sigma.between = 0.5
df.scale.between = 1

B.max = 30

prior.mean.alpha.0  = 0
prior.scale.alpha.0 = 1/10
prior.mean.alpha.1  = 0
prior.scale.alpha.1 = 1/10
```

```
# This list describes the data used by the BUGS script.
data.PBias <- list("y", "se.y", "x", "N", "T",
                   "B.max",
                 "scale.sigma.between",
                 "df.scale.between",
                 "prior.mean.alpha.0",
                 "prior.scale.alpha.0",
                 "prior.mean.alpha.1",
                 "prior.scale.alpha.1")

# List of parameters
par.PBias  <- c("theta.bc",
             "mu",
             "tau",
             "P",
             "T",
             "w",
             "B",
             "alpha.0",
             "alpha.1",
             "K.x")
```

```
# Call with OpenBUGS
PBias.2 <- jags(data.PBias,
             inits = NULL,
             parameters.to.save = par.PBias,
             model = "PBias.bugs",
             n.chains = 4,
             n.iter = 50000,
             n.thin = 4,
             n.burnin = 20000,
             DIC = TRUE)
```

```
## Compiling model graph
##    Resolving undeclared variables
##    Allocating nodes
## Graph information:
##    Observed stochastic nodes: 33
##    Unobserved stochastic nodes: 97
##    Total graph size: 472
##
## Initializing model
```

Posterior intervals of α_1 and $alpha_2$:

```
attach.jags(PBias.2, overwrite = TRUE)

round(mean(alpha.0), 3)
```

```
## [1] -4.862
```

```
round(quantile(alpha.0, probs= c(0.025,  0.975)), 3)
```

```
##   2.5%  97.5%
## -9.585 -0.028
```

```
round(mean(alpha.1), 3)
```

```
## [1] 0.129
```

```
round(quantile(alpha.1, probs= c(0.025,  0.975)), 3)
```

```
##   2.5%  97.5%
## -0.023  0.319
```

Probability that $\alpha_1 > 0$

```
round(sum(alpha.1> 0)/length(alpha.1),3)
```

```
## [1] 0.948
```

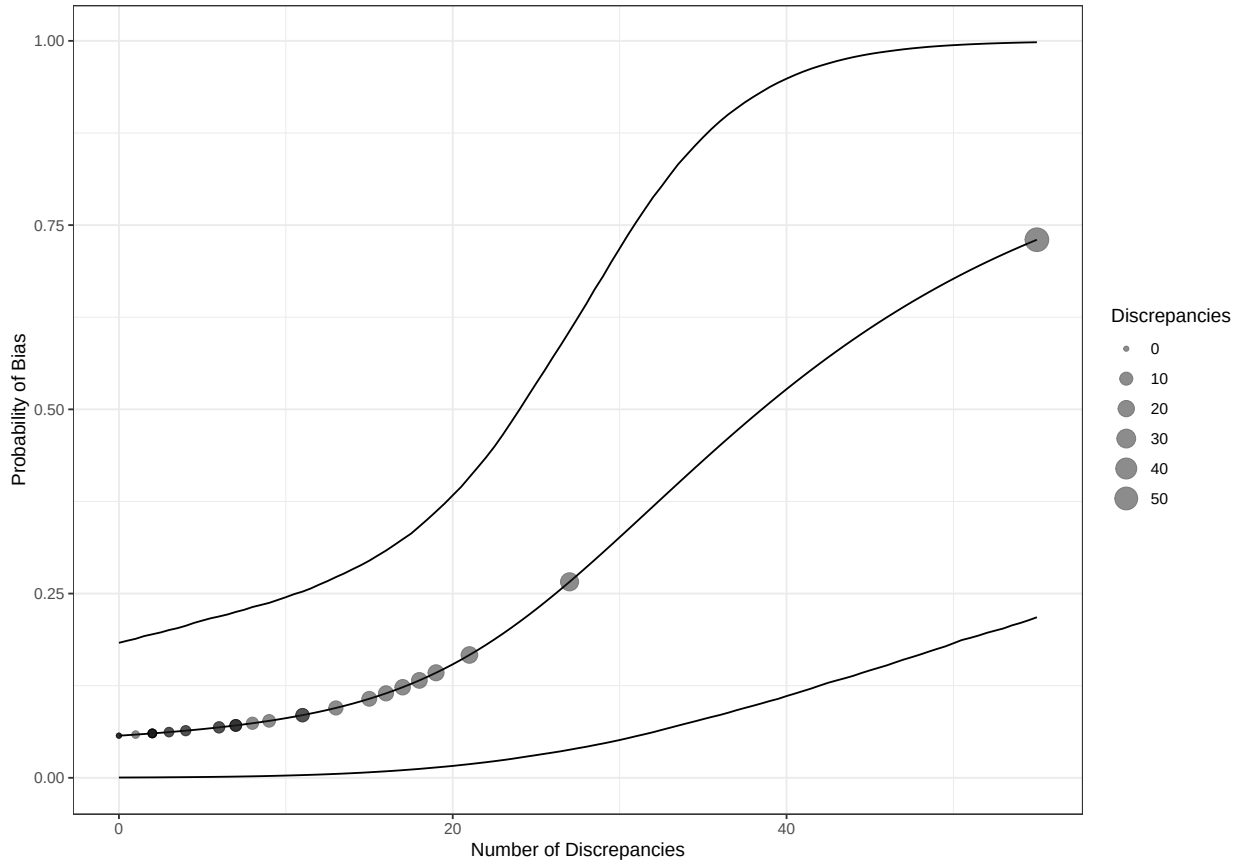
This is not used in the paper, it corresponds to the posterior distribution of a model with a cut-off point on the number of discrepancies. I leave this material as a complementary approach.

```
#quantile(K.x, probs= c(0.025, 0.5, 0.975))
```

```
#attach.jags(PBias.2, overwrite = TRUE)

#df.Kx = data.frame(Kx = K.x)


#ggplot(df.Kx, aes(x = Kx))+
#  geom_histogram(aes(y=..density..), binwidth=1,
#                 colour="black", fill="white") +
#  scale_x_continuous(limits = c(0, 60)) +
  #scale_y_continuous(limits = c(0, 0.45))+
#  xlab("Number of discrepancies")+
#  ylab("Posterior Distribution")+
#  geom_density(alpha=.2, lwd = 1)
#  geom_vline(aes(xintercept=40), lty = 2)
#    theme_bw()
```

```
# Probability as function of discrepancies ....................................

attach.jags(PBias.2, overwrite = TRUE)

inv.logit <- function(x, alpha.0, alpha.1)
{
    1/(1 + exp(-(alpha.0 + alpha.1 *x)))
}


#inv.cloglog <- function(x, alpha.0, alpha.1)
# {
#  1-exp(-exp((alpha.0 + alpha.1 *x)))
#  }


#inv.logit.K <- function(x, alpha.0, alpha.1, K.x)
#    {
#      1/(1 + exp(-(alpha.0 + alpha.1 *I(x>K.x))))
#    }


#inv.cloglog.K <- function(x, alpha.0, alpha.1, K.x)
# {
#  1-exp(-exp((alpha.0 + alpha.1 *I(x>K.x))))
#  }
```

```
p.hat = P[,1:31,2]
p.hat = colMeans(p.hat)

dat.points = data.frame(x.point=stemcells$n.discrep[order(y)], p.hat= p.hat,
                        Discrepancies = stemcells$n.discrep[order(y)])
rm(p.hat)

     x.grid <- seq(0, 55, 0.5)
   post.curv  <- Vectorize(inv.logit, vectorize.args = c("alpha.0", "alpha.1"))

#  post.curv  <- Vectorize(inv.cloglog, vectorize.args = c("alpha.0", "alpha.1"))
#  post.curv  <- Vectorize(inv.logit.K, vectorize.args = c("alpha.0", "alpha.1", "K.x"))
#  post.curv  <- Vectorize(inv.cloglog.K, vectorize.args = c("alpha.0", "alpha.1", "K.x"))
#  post.out <- post.curv(alpha.0, alpha.1, K.x, x = x.grid)

      post.out <- post.curv(alpha.0, alpha.1, x = x.grid)

#    post.CI <- apply(post.out, 1, quantile, prob = c(0.1,0.25, 0.5, 0.75, 0.9), na.rm =T)
#   dat.post <- data.frame(x = x.grid,
#                     y.low = post.CI[1,],
#                     y.q25 = post.CI[2,],
#                     y.med = post.CI[3,],
#                     y.q75 = post.CI[4,],
#                      y.up = post.CI[5,])

     post.CI <- apply(post.out, 1, quantile, prob = c(0.1, 0.9), na.rm =T)


   dat.post <- data.frame(x = x.grid,
                     y.low = post.CI[1,],
                     y.med = apply(post.out, 1, mean, na.rm = T),
                      y.up = post.CI[2,])



  ggplot(data = dat.post, aes_string(x = "x", y = "prob")) +
             geom_line(aes_string(x = "x", y = "y.up"), lty = 1) +
    #        geom_line(aes_string(x = "x", y = "y.q25")) +
             geom_line(aes_string(x = "x", y = "y.med")) +
    #        geom_line(aes_string(x = "x", y = "y.q75")) +
             geom_line(aes_string(x = "x", y = "y.low"), lty = 1) +
             xlab("Number of Discrepancies") +
             ylab("Probability of Bias") +
             geom_point(data = dat.points,
                 aes(x = x.point, y = p.hat, size = Discrepancies),
                 shape = 21, alpha = 0.5,
                 fill = "grey10")+
              theme_bw()
```




```
PBias.2
```

```
## Inference for Bugs model at "PBias.bugs", fit using jags,
##  4 chains, each with 50000 iterations (first 20000 discarded), n.thin = 4
##  n.sims = 30000 iterations saved
##              mu.vect sd.vect    2.5%     25%     50%     75%   97.5%  Rhat
## B             10.085   5.650   1.098   5.945   9.420  13.378  23.838 1.001
## K.x           29.951  17.379   1.548  14.834  29.770  45.138  58.533 1.001
## P[1,1]         0.877   0.156   0.405   0.850   0.936   0.974   0.997 1.018
## P[2,1]         0.929   0.137   0.492   0.942   0.983   0.995   1.000 1.028
## P[3,1]         0.929   0.137   0.492   0.942   0.983   0.995   1.000 1.028
## P[4,1]         0.936   0.134   0.505   0.955   0.988   0.997   1.000 1.029
## P[5,1]         0.936   0.134   0.505   0.955   0.988   0.997   1.000 1.029
## P[6,1]         0.833   0.176   0.331   0.774   0.895   0.956   0.994 1.011
## P[7,1]         0.905   0.145   0.451   0.899   0.962   0.986   0.998 1.024
## P[8,1]         0.868   0.160   0.389   0.834   0.927   0.970   0.996 1.016
## P[9,1]         0.885   0.152   0.417   0.864   0.943   0.978   0.997 1.020
## P[10,1]        0.943   0.133   0.507   0.967   0.993   0.998   1.000 1.029
## P[11,1]        0.734   0.225   0.158   0.613   0.802   0.912   0.986 1.004
## P[12,1]        0.915   0.141   0.471   0.917   0.970   0.990   0.999 1.026
## P[13,1]        0.940   0.133   0.505   0.962   0.991   0.998   1.000 1.029
## P[14,1]        0.938   0.134   0.505   0.958   0.990   0.997   1.000 1.029
## P[15,1]        0.943   0.133   0.507   0.967   0.993   0.998   1.000 1.029
## P[16,1]        0.943   0.133   0.507   0.967   0.993   0.998   1.000 1.029
## P[17,1]        0.940   0.133   0.505   0.962   0.991   0.998   1.000 1.029
## P[18,1]        0.938   0.134   0.505   0.958   0.990   0.997   1.000 1.029
## P[19,1]        0.926   0.138   0.486   0.937   0.980   0.994   0.999 1.028
## P[20,1]        0.932   0.136   0.498   0.947   0.985   0.995   1.000 1.028
## P[21,1]        0.940   0.133   0.505   0.962   0.991   0.998   1.000 1.029
## P[22,1]        0.857   0.164   0.371   0.815   0.918   0.966   0.995 1.014
```

```
## P[23,1]        0.929   0.137   0.492   0.942   0.983   0.995   1.000 1.028
## P[24,1]        0.915   0.141   0.471   0.917   0.970   0.990   0.999 1.026
## P[25,1]        0.893   0.149   0.430   0.877   0.950   0.981   0.998 1.021
## P[26,1]        0.932   0.136   0.498   0.947   0.985   0.995   1.000 1.028
## P[27,1]        0.923   0.139   0.481   0.931   0.977   0.993   0.999 1.027
## P[28,1]        0.940   0.133   0.505   0.962   0.991   0.998   1.000 1.029
## P[29,1]        0.940   0.133   0.505   0.962   0.991   0.998   1.000 1.029
## P[30,1]        0.941   0.133   0.507   0.965   0.992   0.998   1.000 1.029
## P[31,1]        0.270   0.298   0.000   0.019   0.138   0.464   0.949 1.001
## P[1,2]         0.123   0.156   0.003   0.026   0.064   0.150   0.595 1.002
## P[2,2]         0.071   0.137   0.000   0.005   0.017   0.058   0.508 1.002
## P[3,2]         0.071   0.137   0.000   0.005   0.017   0.058   0.508 1.002
## P[4,2]         0.064   0.134   0.000   0.003   0.012   0.045   0.495 1.002
## P[5,2]         0.064   0.134   0.000   0.003   0.012   0.045   0.495 1.002
## P[6,2]         0.167   0.176   0.006   0.044   0.105   0.226   0.669 1.002
## P[7,2]         0.095   0.145   0.002   0.014   0.038   0.101   0.549 1.002
## P[8,2]         0.132   0.160   0.004   0.030   0.073   0.166   0.611 1.002
## P[9,2]         0.115   0.152   0.003   0.022   0.057   0.136   0.583 1.002
## P[10,2]        0.057   0.133   0.000   0.002   0.007   0.033   0.493 1.002
## P[11,2]        0.266   0.225   0.014   0.088   0.198   0.387   0.842 1.002
## P[12,2]        0.085   0.141   0.001   0.010   0.030   0.083   0.529 1.002
## P[13,2]        0.060   0.133   0.000   0.002   0.009   0.038   0.495 1.002
## P[14,2]        0.062   0.134   0.000   0.003   0.010   0.042   0.495 1.002
## P[15,2]        0.057   0.133   0.000   0.002   0.007   0.033   0.493 1.002
## P[16,2]        0.057   0.133   0.000   0.002   0.007   0.033   0.493 1.002
## P[17,2]        0.060   0.133   0.000   0.002   0.009   0.038   0.495 1.002
## P[18,2]        0.062   0.134   0.000   0.003   0.010   0.042   0.495 1.002
## P[19,2]        0.074   0.138   0.001   0.006   0.020   0.063   0.514 1.002
## P[20,2]        0.068   0.136   0.000   0.005   0.015   0.053   0.502 1.002
## P[21,2]        0.060   0.133   0.000   0.002   0.009   0.038   0.495 1.002
## P[22,2]        0.143   0.164   0.005   0.034   0.082   0.185   0.629 1.002
## P[23,2]        0.071   0.137   0.000   0.005   0.017   0.058   0.508 1.002
## P[24,2]        0.085   0.141   0.001   0.010   0.030   0.083   0.529 1.002
## P[25,2]        0.107   0.149   0.002   0.019   0.050   0.123   0.570 1.002
## P[26,2]        0.068   0.136   0.000   0.005   0.015   0.053   0.502 1.002
## P[27,2]        0.077   0.139   0.001   0.007   0.023   0.069   0.519 1.002
## P[28,2]        0.060   0.133   0.000   0.002   0.009   0.038   0.495 1.002
## P[29,2]        0.060   0.133   0.000   0.002   0.009   0.038   0.495 1.002
## P[30,2]        0.059   0.133   0.000   0.002   0.008   0.035   0.493 1.002
## P[31,2]        0.730   0.298   0.051   0.536   0.862   0.981   1.000 1.001
## T[1]           1.000   0.000   1.000   1.000   1.000   1.000   1.000 1.000
## T[2]           1.024   0.154   1.000   1.000   1.000   1.000   1.000 1.005
## T[3]           1.023   0.149   1.000   1.000   1.000   1.000   1.000 1.005
## T[4]           1.020   0.141   1.000   1.000   1.000   1.000   1.000 1.006
## T[5]           1.025   0.156   1.000   1.000   1.000   1.000   2.000 1.002
## T[6]           1.050   0.218   1.000   1.000   1.000   1.000   2.000 1.002
## T[7]           1.038   0.191   1.000   1.000   1.000   1.000   2.000 1.006
## T[8]           1.038   0.192   1.000   1.000   1.000   1.000   2.000 1.002
## T[9]           1.026   0.161   1.000   1.000   1.000   1.000   2.000 1.002
## T[10]          1.014   0.117   1.000   1.000   1.000   1.000   1.000 1.018
## T[11]          1.073   0.260   1.000   1.000   1.000   1.000   2.000 1.001
## T[12]          1.023   0.150   1.000   1.000   1.000   1.000   1.000 1.008
## T[13]          1.032   0.176   1.000   1.000   1.000   1.000   2.000 1.009
## T[14]          1.027   0.163   1.000   1.000   1.000   1.000   2.000 1.008
## T[15]          1.029   0.168   1.000   1.000   1.000   1.000   2.000 1.006
## T[16]          1.031   0.173   1.000   1.000   1.000   1.000   2.000 1.009
## T[17]          1.032   0.176   1.000   1.000   1.000   1.000   2.000 1.006
## T[18]          1.046   0.209   1.000   1.000   1.000   1.000   2.000 1.004
## T[19]          1.049   0.217   1.000   1.000   1.000   1.000   2.000 1.003
## T[20]          1.055   0.228   1.000   1.000   1.000   1.000   2.000 1.005
## T[21]          1.045   0.207   1.000   1.000   1.000   1.000   2.000 1.005
## T[22]          1.088   0.283   1.000   1.000   1.000   1.000   2.000 1.002
## T[23]          1.066   0.249   1.000   1.000   1.000   1.000   2.000 1.003
## T[24]          1.105   0.307   1.000   1.000   1.000   1.000   2.000 1.002
## T[25]          1.125   0.331   1.000   1.000   1.000   1.000   2.000 1.003
```

```
## T[26]          1.138   0.344   1.000   1.000   1.000   1.000   2.000 1.003
## T[27]          1.154   0.361   1.000   1.000   1.000   1.000   2.000 1.003
## T[28]          1.113   0.317   1.000   1.000   1.000   1.000   2.000 1.002
## T[29]          1.186   0.389   1.000   1.000   1.000   1.000   2.000 1.002
## T[30]          1.164   0.370   1.000   1.000   1.000   1.000   2.000 1.002
## T[31]          2.000   0.000   2.000   2.000   2.000   2.000   2.000 1.000
## alpha.0       -4.862   2.435  -9.585  -6.430  -4.946  -3.385  -0.028 1.002
## alpha.1        0.129   0.087  -0.023   0.071   0.122   0.180   0.319 1.001
## mu[1]          2.402   0.924   0.290   1.930   2.502   3.001   3.949 1.004
## mu[2]         12.487   5.848   3.465   7.982  12.005  15.970  26.485 1.001
## tau            2.768   0.802   0.855   2.363   2.814   3.264   4.256 1.002
## theta.bc[1]    0.100   2.314  -4.680  -1.363   0.186   1.627   4.503 1.001
## theta.bc[2]   -1.122   1.776  -4.740  -2.300  -1.052   0.109   2.204 1.001
## theta.bc[3]   -0.157   2.120  -4.513  -1.522  -0.087   1.275   3.846 1.001
## theta.bc[4]    0.227   2.052  -3.967  -1.081   0.288   1.583   4.164 1.001
## theta.bc[5]   -1.465   0.633  -2.698  -1.893  -1.469  -1.042  -0.216 1.001
## theta.bc[6]    0.445   2.053  -3.741  -0.875   0.480   1.801   4.401 1.001
## theta.bc[7]    1.615   2.428  -3.204   0.053   1.586   3.185   6.471 1.001
## theta.bc[8]    0.960   1.861  -2.734  -0.256   0.950   2.198   4.588 1.001
## theta.bc[9]    0.309   1.101  -1.860  -0.424   0.316   1.046   2.448 1.001
## theta.bc[10]   0.972   0.928  -0.846   0.344   0.969   1.589   2.816 1.001
## theta.bc[11]   1.216   0.841  -0.439   0.642   1.219   1.785   2.853 1.001
## theta.bc[12]   1.382   0.948  -0.463   0.740   1.378   2.020   3.252 1.001
## theta.bc[13]   2.155   2.406  -2.485   0.559   2.093   3.733   6.999 1.001
## theta.bc[14]   2.201   1.090   0.093   1.456   2.199   2.933   4.342 1.001
## theta.bc[15]   2.457   1.722  -0.848   1.281   2.418   3.606   5.883 1.001
## theta.bc[16]   2.465   1.686  -0.764   1.309   2.446   3.588   5.819 1.001
## theta.bc[17]   2.563   1.915  -1.053   1.244   2.524   3.856   6.456 1.001
## theta.bc[18]   3.291   1.202   0.965   2.472   3.296   4.107   5.645 1.001
## theta.bc[19]   3.293   1.546   0.347   2.223   3.274   4.334   6.361 1.001
## theta.bc[20]   2.908   2.836  -2.469   1.006   2.818   4.708   8.661 1.001
## theta.bc[21]   3.053   2.433  -1.518   1.386   2.973   4.633   7.976 1.001
## theta.bc[22]   3.755   1.421   0.997   2.791   3.738   4.703   6.560 1.001
## theta.bc[23]   3.719   2.430  -0.789   2.046   3.635   5.282   8.732 1.001
## theta.bc[24]   5.002   1.161   2.712   4.222   4.994   5.784   7.308 1.001
## theta.bc[25]   5.069   2.101   1.105   3.632   5.020   6.458   9.320 1.001
## theta.bc[26]   6.531   1.011   4.547   5.850   6.521   7.213   8.524 1.001
## theta.bc[27]   6.741   1.483   3.878   5.731   6.720   7.738   9.714 1.001
## theta.bc[28]   5.642   2.455   1.126   3.973   5.549   7.181  10.772 1.001
## theta.bc[29]   7.944   1.964   4.288   6.596   7.874   9.210  12.016 1.001
## theta.bc[30]   7.260   2.336   3.010   5.665   7.134   8.715  12.302 1.001
## theta.bc[31]  13.931   1.024  11.925  13.242  13.930  14.618  15.951 1.001
## w[1,1]         1.000   0.000   1.000   1.000   1.000   1.000   1.000 1.000
## w[2,1]         0.334   0.298   0.001   0.065   0.251   0.563   0.951 1.001
## w[1,2]         1.000   0.000   1.000   1.000   1.000   1.000   1.000 1.000
## w[2,2]         0.326   0.296   0.001   0.060   0.238   0.545   0.949 1.001
## w[1,3]         1.000   0.000   1.000   1.000   1.000   1.000   1.000 1.000
## w[2,3]         0.329   0.297   0.000   0.061   0.244   0.554   0.951 1.001
## w[1,4]         1.000   0.000   1.000   1.000   1.000   1.000   1.000 1.000
## w[2,4]         0.329   0.296   0.001   0.063   0.243   0.552   0.948 1.001
## w[1,5]         1.000   0.000   1.000   1.000   1.000   1.000   1.000 1.000
## w[2,5]         0.330   0.299   0.001   0.060   0.245   0.561   0.949 1.001
## w[1,6]         1.000   0.000   1.000   1.000   1.000   1.000   1.000 1.000
## w[2,6]         0.329   0.297   0.001   0.061   0.242   0.554   0.950 1.001
## w[1,7]         1.000   0.000   1.000   1.000   1.000   1.000   1.000 1.000
## w[2,7]         0.335   0.298   0.001   0.065   0.253   0.565   0.951 1.001
## w[1,8]         1.000   0.000   1.000   1.000   1.000   1.000   1.000 1.000
## w[2,8]         0.335   0.298   0.001   0.064   0.251   0.560   0.956 1.002
## w[1,9]         1.000   0.000   1.000   1.000   1.000   1.000   1.000 1.000
## w[2,9]         0.330   0.297   0.001   0.062   0.248   0.554   0.951 1.002
## w[1,10]        1.000   0.000   1.000   1.000   1.000   1.000   1.000 1.000
## w[2,10]        0.335   0.299   0.001   0.063   0.250   0.564   0.953 1.001
## w[1,11]        1.000   0.000   1.000   1.000   1.000   1.000   1.000 1.000
## w[2,11]        0.331   0.296   0.001   0.064   0.244   0.560   0.948 1.003
## w[1,12]        1.000   0.000   1.000   1.000   1.000   1.000   1.000 1.000
```

```
## w[2,12]        0.332   0.298   0.001   0.064   0.246   0.559   0.950 1.001
## w[1,13]        1.000   0.000   1.000   1.000   1.000   1.000   1.000 1.000
## w[2,13]        0.333   0.298   0.001   0.063   0.249   0.562   0.949 1.001
## w[1,14]        1.000   0.000   1.000   1.000   1.000   1.000   1.000 1.000
## w[2,14]        0.334   0.297   0.001   0.065   0.252   0.563   0.950 1.001
## w[1,15]        1.000   0.000   1.000   1.000   1.000   1.000   1.000 1.000
## w[2,15]        0.338   0.298   0.001   0.067   0.257   0.571   0.950 1.001
## w[1,16]        1.000   0.000   1.000   1.000   1.000   1.000   1.000 1.000
## w[2,16]        0.333   0.298   0.001   0.063   0.249   0.562   0.951 1.001
## w[1,17]        1.000   0.000   1.000   1.000   1.000   1.000   1.000 1.000
## w[2,17]        0.337   0.298   0.001   0.066   0.256   0.564   0.953 1.001
## w[1,18]        1.000   0.000   1.000   1.000   1.000   1.000   1.000 1.000
## w[2,18]        0.334   0.297   0.001   0.066   0.251   0.563   0.948 1.001
## w[1,19]        1.000   0.000   1.000   1.000   1.000   1.000   1.000 1.000
## w[2,19]        0.335   0.298   0.001   0.063   0.253   0.566   0.950 1.001
## w[1,20]        1.000   0.000   1.000   1.000   1.000   1.000   1.000 1.000
## w[2,20]        0.333   0.297   0.001   0.063   0.251   0.561   0.949 1.001
## w[1,21]        1.000   0.000   1.000   1.000   1.000   1.000   1.000 1.000
## w[2,21]        0.338   0.297   0.001   0.067   0.259   0.569   0.951 1.001
## w[1,22]        1.000   0.000   1.000   1.000   1.000   1.000   1.000 1.000
## w[2,22]        0.335   0.296   0.001   0.068   0.255   0.561   0.950 1.001
## w[1,23]        1.000   0.000   1.000   1.000   1.000   1.000   1.000 1.000
## w[2,23]        0.336   0.298   0.001   0.066   0.253   0.566   0.950 1.001
## w[1,24]        1.000   0.000   1.000   1.000   1.000   1.000   1.000 1.000
## w[2,24]        0.340   0.299   0.001   0.067   0.260   0.574   0.955 1.001
## w[1,25]        1.000   0.000   1.000   1.000   1.000   1.000   1.000 1.000
## w[2,25]        0.344   0.298   0.001   0.072   0.266   0.576   0.952 1.002
## w[1,26]        1.000   0.000   1.000   1.000   1.000   1.000   1.000 1.000
## w[2,26]        0.342   0.298   0.001   0.072   0.264   0.572   0.952 1.001
## w[1,27]        1.000   0.000   1.000   1.000   1.000   1.000   1.000 1.000
## w[2,27]        0.346   0.298   0.001   0.074   0.271   0.578   0.954 1.001
## w[1,28]        1.000   0.000   1.000   1.000   1.000   1.000   1.000 1.000
## w[2,28]        0.345   0.300   0.001   0.071   0.265   0.583   0.953 1.001
## w[1,29]        1.000   0.000   1.000   1.000   1.000   1.000   1.000 1.000
## w[2,29]        0.342   0.297   0.001   0.071   0.266   0.573   0.949 1.001
## w[1,30]        1.000   0.000   1.000   1.000   1.000   1.000   1.000 1.000
## w[2,30]        0.341   0.298   0.001   0.069   0.262   0.573   0.953 1.002
## w[1,31]        1.000   0.000   1.000   1.000   1.000   1.000   1.000 1.000
## w[2,31]        0.353   0.292   0.004   0.092   0.279   0.577   0.954 1.002
## deviance     132.421   8.137 118.396 126.665 131.802 137.533 149.953 1.001
##              n.eff
## B            19000
## K.x          13000
## P[1,1]        1300
## P[2,1]        1100
## P[3,1]        1100
## P[4,1]        1100
## P[5,1]        1100
## P[6,1]        1800
## P[7,1]        1100
## P[8,1]        1400
## P[9,1]        1300
## P[10,1]       1100
## P[11,1]       3500
## P[12,1]       1100
## P[13,1]       1100
## P[14,1]       1100
## P[15,1]       1100
## P[16,1]       1100
## P[17,1]       1100
## P[18,1]       1100
## P[19,1]       1100
## P[20,1]       1100
## P[21,1]       1100
## P[22,1]       1500
```

```
## P[23,1]        1100
## P[24,1]        1100
## P[25,1]        1200
## P[26,1]        1100
## P[27,1]        1100
## P[28,1]        1100
## P[29,1]        1100
## P[30,1]        1100
## P[31,1]       13000
## P[1,2]         2600
## P[2,2]         2400
## P[3,2]         2400
## P[4,2]         2500
## P[5,2]         2500
## P[6,2]         2900
## P[7,2]         2400
## P[8,2]         2600
## P[9,2]         2500
## P[10,2]        2600
## P[11,2]        4100
## P[12,2]        2400
## P[13,2]        2500
## P[14,2]        2500
## P[15,2]        2600
## P[16,2]        2600
## P[17,2]        2500
## P[18,2]        2500
## P[19,2]        2400
## P[20,2]        2400
## P[21,2]        2500
## P[22,2]        2700
## P[23,2]        2400
## P[24,2]        2400
## P[25,2]        2500
## P[26,2]        2400
## P[27,2]        2400
## P[28,2]        2500
## P[29,2]        2500
## P[30,2]        2600
## P[31,2]        9500
## T[1]              1
## T[2]           3900
## T[3]           4500
## T[4]           4000
## T[5]           8500
## T[6]           6200
## T[7]           2400
## T[8]           7000
## T[9]           9100
## T[10]          2000
## T[11]         30000
## T[12]          2700
## T[13]          1800
## T[14]          2300
## T[15]          3200
## T[16]          1900
## T[17]          2800
## T[18]          2900
## T[19]          3300
## T[20]          2200
## T[21]          2300
## T[22]          3900
## T[23]          2600
## T[24]          2300
## T[25]          1900
```

```
## T[26]         1700
## T[27]         1700
## T[28]         2200
## T[29]         2300
## T[30]         1800
## T[31]            1
## alpha.0       2200
## alpha.1       4800
## mu[1]         2200
## mu[2]         4500
## tau           3300
## theta.bc[1]  30000
## theta.bc[2]  20000
## theta.bc[3]  30000
## theta.bc[4]  30000
## theta.bc[5]  30000
## theta.bc[6]  30000
## theta.bc[7]  22000
## theta.bc[8]  30000
## theta.bc[9]  30000
## theta.bc[10] 30000
## theta.bc[11] 20000
## theta.bc[12] 30000
## theta.bc[13] 15000
## theta.bc[14] 30000
## theta.bc[15] 30000
## theta.bc[16] 30000
## theta.bc[17] 20000
## theta.bc[18] 20000
## theta.bc[19] 30000
## theta.bc[20] 25000
## theta.bc[21] 29000
## theta.bc[22] 12000
## theta.bc[23] 30000
## theta.bc[24] 23000
## theta.bc[25] 30000
## theta.bc[26] 21000
## theta.bc[27]  7200
## theta.bc[28] 30000
## theta.bc[29] 23000
## theta.bc[30] 30000
## theta.bc[31] 13000
## w[1,1]           1
## w[2,1]       14000
## w[1,2]           1
## w[2,2]        9000
## w[1,3]           1
## w[2,3]       30000
## w[1,4]           1
## w[2,4]       30000
## w[1,5]           1
## w[2,5]       29000
## w[1,6]           1
## w[2,6]        6900
## w[1,7]           1
## w[2,7]        8700
## w[1,8]           1
## w[2,8]        6400
## w[1,9]           1
## w[2,9]        4100
## w[1,10]          1
## w[2,10]       8100
## w[1,11]          1
## w[2,11]       4600
## w[1,12]          1
```

```
## w[2,12]      11000
## w[1,13]          1
## w[2,13]       7300
## w[1,14]          1
## w[2,14]      27000
## w[1,15]          1
## w[2,15]      30000
## w[1,16]          1
## w[2,16]       5800
## w[1,17]          1
## w[2,17]      30000
## w[1,18]          1
## w[2,18]      30000
## w[1,19]          1
## w[2,19]      30000
## w[1,20]          1
## w[2,20]       5800
## w[1,21]          1
## w[2,21]      12000
## w[1,22]          1
## w[2,22]      30000
## w[1,23]          1
## w[2,23]      30000
## w[1,24]          1
## w[2,24]      13000
## w[1,25]          1
## w[2,25]       4000
## w[1,26]          1
## w[2,26]      30000
## w[1,27]          1
## w[2,27]       6700
## w[1,28]          1
## w[2,28]      20000
## w[1,29]          1
## w[2,29]      14000
## w[1,30]          1
## w[2,30]       3400
## w[1,31]          1
## w[2,31]       2200
## deviance     11000
##
## For each parameter, n.eff is a crude measure of effective sample size,
## and Rhat is the potential scale reduction factor (at convergence, Rhat=1).
##
## DIC info (using the rule, pD = var(deviance)/2)
## pD = 33.1 and DIC = 165.5
## DIC is an estimate of expected predictive error (lower deviance is better).
```